RTSP实时传输图像

RTSP实时传输图像
- 例程讲解
  - 简介
  - 准备
  - 启动例程
  - 打开播放器
- 例程代码

RTSP (Real Time Streaming Protocol) 简介
- 什么是RTSP?
- 主要特点
- 常见应用场景
- RTSP vs 其他流媒体协议
  - 协议对比
- RTSP工作流程
- RTSP架构
- RTSP传输模式
- 优势
- 劣势

例程讲解

简介

本节我们演示一下把K230捕获到的图像以RTSP流的形式推流。我们可以用支持播放RTSP流的视频播放器播放实时传输的画面。

> 注：K230上的WIFI芯片性能有限，请在靠近WIFI信号热点的地方使用图传

> *本例程为实验性内容，在不同网络环境下有概率出现无法高质量运行的可能性

> 如果想要提升视频流畅度，可以修改这一行代码的bit_rate参数，bit_rate的建议取值范围为 10 ~ 300
>
> **bit_rate 值越小流畅度越好，但是清晰度越低**
>
> ```
> # 创建编码器 / Create encoder
>     chnAttr = ChnAttrStr(self.encoder.PAYLOAD_TYPE_H264,
> self.encoder.H264_PROFILE_MAIN, width, height, bit_rate=10,
> dst_frame_rate=5, src_frame_rate=5)
> ```

准备

我们可以下载一个支持播放RTSP流的播放器，Windows系统可选择的有

1. VLC 播放器 （配套资料中的 vlc-3.0.21-win64.exe，双击即可安装）

   优点：兼容性好、功能强大

   缺点：配置繁多且复杂，如果不会配置的话显示效果会比较差

2. YAHBOOM Rtsp Player（已内置在配套小工具程序中）

   优点：无需配置，一键使用，低延迟

缺点：体积较大

建议两个都下载，一般情况下使用YAHBOOM Rtsp Player，如果YAHBOOM Rtsp Player出现异常情况，则使用Vlc播放器调试

> 其它系统下暂未做过尝试，大家可以自行搜索xxx系统支持的RTSP播放器。目前已测试过Vlc for Android可以在安卓手机上播放RTSP视频，但是延迟很高，暂时没找到配置的方法

启动例程

> RTSP图传对网络质量要求很高，请尽量选取一个路由器挂载设备少、信号强劲的WIFI
>
> **针对图传功能，建议使用手机开启热点（可以不开启WIFI和流量），选择2.4GHz的频率，K230放在手机旁边使用**
>
> 图传出现画面卡顿属正常情况，可以从下面几个方向优化：
>
> 1. 检查WIFI是否使用的是2.4Ghz。K230上面的WIFI芯片仅支持2.4Ghz的无线信号，5Ghz或2.4/5Ghz混合信号都不支持
> 2. 检查一下路由器是否挂载了过多的设备

请将K230和电脑置于统一局域网中（连接同一个WIFI），修改这一行的参数为WIFI名称和密码

```
# 我连接的WIFI名称为test，密码12345678
isConnected = Connect_WIFI("test","12345678")
```

然后点击运行按钮，等待连接WIFI并启动RTSP服务器

> 常见问题:
>
> 1. 点击运行按钮后无反应
>
>   解决办法：K230 断电，等待5s后再插上线运行。RTSP例程运行每次结束后都需要重启K230。
>
> 2. 报错：run connect failed.
>
>   解决办法：K230 断电，等待5s后再插上线运行。正常情况下最多重复三次这个操作就能恢复正常，
>
>   如果一直报错run connect failed. 大概率是WIFI芯片损坏

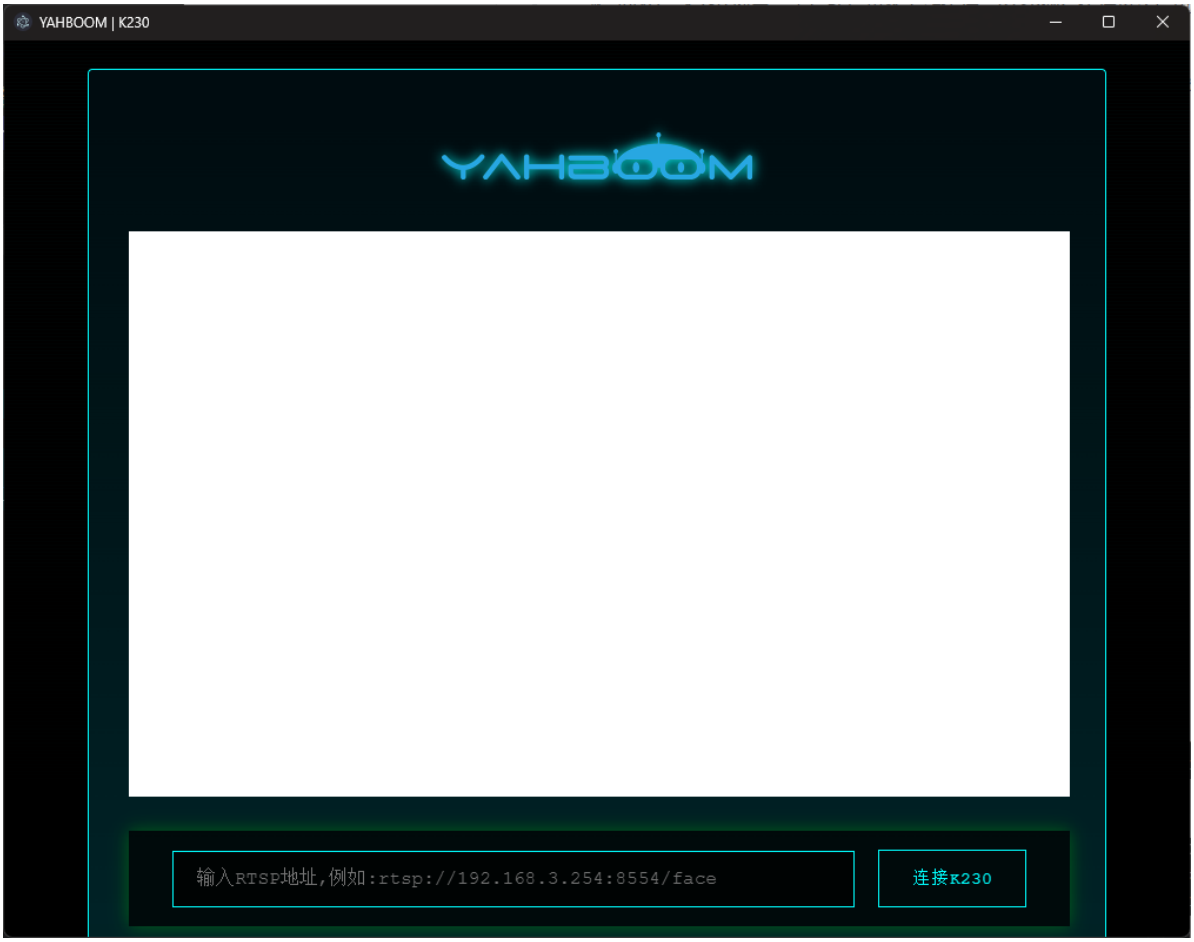
如果正常启动，K230串行终端会输出下面的内容

```
[WIFI] 连接网络中 ...
[WIFI] 连接网络成功
[RTSP] 启动中 ..
find sensor gc2093_csi2, type 24, output 1920x1080@60
buffer pool :  3
sensor(0), mode 0, buffer_num 4, buffer_size 0
[RTSP] 启动成功, 地址: rtsp://192.168.207.22:8554/video
```

这个地址就是你的RTSP推流的地址。

打开播放器

以YAHBOOM Rtsp Player为例，双击运行程序，界面如下：

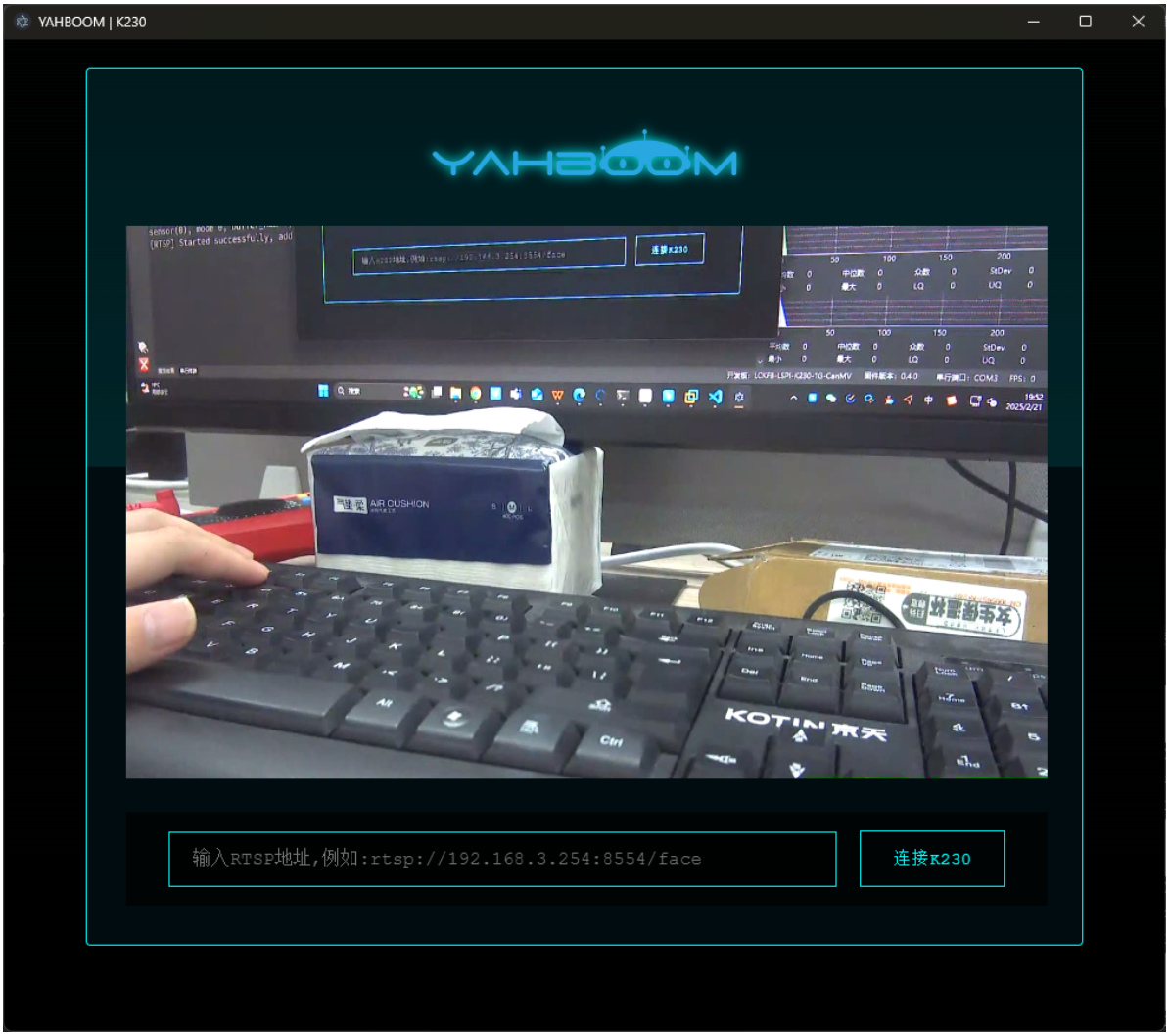


> 注：实际界面可能稍有不同，但是总体功能是一样的



点击连接后，会加载一段时间，加载速度与网速有关

> 若等待20秒后依旧无反应，请尝试使用 VLC Player播放器播放。如果还是没反应，请拔下USB线断电，10秒后再插上，重新运行该例程代码



例程代码

完整代码请参考文件：【源码汇总 / 11.Network / 07.rtsp / rtsp_no_ai.py】

1. 整体结构与功能：

- 代码主要实现了视频采集、编码和 RTSP 推流功能
- 包含了 WiFi 连接模块和 RTSP 服务器类两个主要部分
- 使用多线程处理视频流数据

2. 关键类和方法：

RtspServer 类的核心成员：

```
def __init__(self):
    self.session_name = session_name  # 会话名称
    self.video_type = video_type      # 视频类型(H.264/H.265)
    self.enable_audio = enable_audio  # 是否启用音频
    self.port = port                  # RTSP 端口号
    self.rtspserver = mm.rtsp_server() # RTSP服务器实例
    self.venc_chn = VENC_CHN_ID_0     # 视频编码通道
```

主要方法包括：

- start(): 启动服务器
- stop(): 停止服务器
- _init_stream(): 初始化视频流
- _do_rtsp_stream(): 处理视频流的核心线程

3. 视频处理流程：

a) 采集阶段：

```
# 通过sensor采集图像
rtsp_show_img = self.sensor.snapshot()

# 设置帧信息
frame_info.v_frame.width = rtsp_show_img.width()
frame_info.v_frame.height = rtsp_show_img.height()
frame_info.v_frame.pixel_format = Sensor.YUV420SP
```

b) 编码阶段：

```
# 发送帧到编码器
self.encoder.SendFrame(self.venc_chn, frame_info)
# 获取编码后的流
self.encoder.GetStream(self.venc_chn, streamData)
```

c) 推流阶段：

```
# 将编码数据发送到RTSP服务器
self.rtspserver.rtspserver_sendvideodata(self.session_name,stream_data,
streamData.data_size[pack_idx],1000)
```

4. 优化设计：

- 使用多线程处理视频流，避免阻塞
- 积极调用垃圾回收
- 设置了视频缓冲区
- 支持不同分辨率的图像处理

5. 错误处理：

- WiFi 连接失败处理
- 视频流异常处理
- 资源释放机制

6. 使用示例：

```
if __name__ == "__main__":
    # 连接WiFi
    isConnected = Connect_WIFI("test","password")

    # 创建并启动RTSP服务器
    rtspserver = RtspServer()
    rtspserver.start()

    # 获取并打印RTSP地址
    rtsp_address = rtspserver.get_rtsp_url()
```

RTSP (Real Time Streaming Protocol) 简介

什么是RTSP?

RTSP是一种用于控制流媒体服务器的网络应用层协议，由RFC 2326定义。它建立并控制一个或多个时间同步的连续媒体流。

主要特点

- 实时性强
- 支持点播和直播
- 支持流媒体控制（播放、暂停、快进等）
- 工作在应用层，默认使用554端口

常见应用场景

1. **视频监控系统**
   - 安防摄像头
   - 工业监控
   - 交通监控
2. **流媒体直播**
   - 视频会议
   - 在线教育
   - 远程医疗

RTSP vs 其他流媒体协议

协议对比

协议	延迟	可靠性	交互性	应用场景
RTSP	低	中	高	视频监控、点播
HLS	高	高	低	直播、点播
RTMP	低	中	中	直播推流
WebRTC	极低	中	高	实时通讯

RTSP工作流程

RTSP架构

RTSP传输模式

优势

1. 实时性好，延迟低
2. 支持流媒体控制功能
3. 带宽占用相对较小
4. 适合点播场景

劣势

1. 需要专门的流媒体服务器
2. 可能被防火墙阻止
3. 不支持自适应码率
4. Web浏览器原生不支持